

Bundle-Analyse: clicksolutions/contao-login-screen

Stand: 08.10.2026 · Methodik: Skills *php-pro* und *php-best-practices* · PHP 8.5.11 (Runtime), Anforderung ^{^8.3} · Contao 5.7

1. Zusammenfassung

Das Bundle ist klein (358 Zeilen, 1 Service) und sauber aufgebaut: `strict_types` in jeder Datei, Constructor Promotion mit `readonly`, Return-Types überall, HTML-Escaping des Freitexts, bewusst keine Template-Kopie von `be_login`. Die Syntaxprüfung (`php -l`) ist für alle Dateien fehlerfrei.

Hauptschwächen: **1 echter Funktionsfehler** (Frontend-Link geht bei fehlendem Anker verloren), **fehlende Ausfallsicherheit** des Login-Screens bei Exceptions im Hook, **harte Kopplung an Contao-Statics** (PHPStan Level 9: 6 Fehler), CSS wird global statt nur am Login geladen sowie fehlende Übersetzungen/Tests/Tooling.

Schwere	Anzahl
Hoch	2
Mittel	5
Niedrig	7
Info / positiv	siehe Abschnitt 5

Messergebnisse

Prüfung	Ergebnis
<code>php -l</code> (6 PHP-Dateien)	OK
PHPStan Level 9 (<code>src/</code>)	6 Fehler: 4× <code>property.notFound</code> (<code>PageModel::\$cs_cls_*</code>), 2× <code>argument.type</code> (<code>fromUuid()</code> erwartet <code>string</code> , bekommt <code>string null</code>)
<code>php-cs-fixer</code> @PSR12 (dry-run)	1 Datei mit Abweichungen: <code>{}</code> des Konstruktors, Einrückung von <code>setSizeItems([...])</code>
Tests / CI / Linter-Config	nicht vorhanden

2. Befunde

H1 Frontend-Link geht verloren, wenn `</main>` fehlt

OutputBackendTemplateListener.php:76-86

`moveFrontendLinkIntoMain()` entfernt den `fe-link`-Absatz per `str_replace()` bevor `insertBefore()` den Anker sucht. Fehlt `</main>` (geänderte Core-Markup), gibt `insertBefore()` den bereits bereinigten Buffer zurück – der Link ist weg, obwohl das Bundle „im Zweifel nichts ändern“ verspricht.

```
$stripped = str_replace($m[0], '', $buffer);
$result   = $this->insertBefore($stripped, '</main>', $link);
return $result === $stripped ? $buffer : $result;    // oder Anker vorher prüfen
```

H2 Keine Fehlerabsicherung im Hook (Login darf nie brechen)

OutputBackendTemplateListener.php:38-74

Der Hook läuft vor jedem Backend-Login. DB-Fehler (z.B. Spalten `cs_cls_*` vor `contao:migrate` nicht vorhanden), Image-Studio-Exceptions (defekte/zu große Datei, fehlende GD/Imagick-Features) oder `preg_*`-Fehler führen zu HTTP 500 – und damit zum Aussperren aller Redakteure. Die Projektgeschichte (alter `be_login`-Override brach Passkey/SSO) zeigt, wie sensibel das ist.

```
try { ...Logik... } catch (\Throwable $e) {
    $this->logger->error('Login screen: ' . $e->getMessage(), ['exception' => $e]);
    return $buffer;
}
```

M1 Statische Contao-Aufrufe statt Framework-Adapter / Request (PHPStan 6 Fehler)

:126, :133, :147, :153, :190, :202, :205, :66

`FilesModel`, `PageModel`, `Environment`, `StringUtil`, `new FrontendTemplate()` werden statisch genutzt. Contao-Konvention (und DIP/Testbarkeit): `ContaoFramework` injizieren, `$this->framework->initialize()` bzw. `getAdapter()`, `Host` aus `RequestStack` statt `Environment::get('host')`. Die PHPStan-Fehler entstehen durch fehlende Typinformation der Magic-Properties; sie lassen sich sauber über eine kleine Typ-Hilfe lösen (z.B. `$page->row()['cs_cls_logo'] ?? null` oder `PHPDoc-Stub/Neon-universalObjectCratesClasses`). Außerdem: `fromUuid()` bekommt `$logo->uuid` – besser `fromFilesModel($logo)`, das spart die zweite Lookup-Query und den Typfehler.

M2 CSS wird in jedem Backend-Request (und ggf. Frontend) geladen

contao/config/config.php:15

`$GLOBALS['TL_CSS']` wird in `config.php` bei jedem Request gesetzt; in Contao 5.7 wertet auch `PageRegular` dieses Global aus. Die Regeln sind auf `body.cs_cls_login` gescoped, der Request ist aber auf allen anderen Backend-Seiten überflüssig. Besser: `<link>` im Hook nur auf `be_login*` einfügen (zusätzlicher Anker `</head>`) oder Global nur im Backend-Scope setzen, dann entfällt `config.php` komplett.

M3 Keine Übersetzungen in Symfony-Format, nur Deutsch

contao/languages/de/tl_page.php

Für englische Backend-Nutzer erscheinen die Feldnamen roh (`cs_cls_logo`). Die Schwesterbundles nutzen bereits `translations/contao_tl_page.{de,en}.yaml`; Legacy-`TL_LANG` ist ab Contao 6 nicht mehr Zielformat. Empfehlung: Migration + `en` ergänzen.

M4 Fragile String-/Regex-Injektion

:56, :62, :79, :91

(a) `/\bclass="..."` trifft auch `data-class="..."` (`\b` zwischen `-` und `c`) und ignoriert `class='...'`; (b) `'<h1>'` matcht nur exakt ohne Attribute; (c) `insertBefore('</main>')` nutzt das erste Vorkommen.

Aktuell passt es zum Core-Markup, jedes Core-Update kann es still brechen (nur Warnung im Log).

Empfehlung: `\bclass` durch `(?![\w-])class` ersetzen, optional `DOMDocument` / `Symfony\DomCrawler` oder Anker-Smoke-Test (siehe T1), und Warnungen nur 1× pro Request loggen.

M5 Veraltete/unnötige Dev-Abhängigkeit, kein Tooling

`composer.json`

`doctrine/doctrine-cache-bundle ^1.3` ist abgekündigt und wird nicht verwendet → entfernen. Es fehlen `phpstan/phpstan`, `contao/easy-coding-standard` (Contao-Coding-Standard, PSR-12-basiert) und `phpunit/phpunit` sowie die jeweiligen Konfigurationsdateien/Composer-Scripts (`"scripts": {"stan": ...}`).

Niedrig

#	Stelle	Befund / Empfehlung
L1	<code>OutputBackendTemplateListener</code> (class)	Nicht <code>final</code> , aber <code>protected</code> -Methoden ohne Erweiterungskonzept. <code>final class</code> verwenden, Methoden <code>private</code> (php-best-practices: <code>solid-ocp</code> , Kapselung).
L2	<code>config/services.yaml</code>	Hook per YAML-Tag; idiomatischer ist <code># [AsHook('outputBackendTemplate')]</code> an der Klasse (Contao ≥ 5.0, vom Core-Extension registriert) plus <code>autowire: true</code> , <code>bind: \$projectDir: '%kernel.project_dir%'</code> . Weniger Konfiguration, Typänderungen brechen nicht still.
L3	Zeile 66	<code>if (\$text = trim(...))</code> wertet <code>"0"</code> als leer. <code>' ' !== \$text</code> verwenden. <code>strip_tags</code> auf bereits entity-kodiertem Contao-Input ist faktisch wirkungslos (kein Schaden; <code>double_encode=false</code> ist hier korrekt gewählt).
L4	Zeilen 126, 153	<code>FilesModel::findByUuid(null)</code> bei leerem Feld: Frühabbruch <code>if (!\$value) return null;</code> einbauen. Bei <code>cs_cls_bg_image</code> können deserialisierte Einträge ungültig sein → Filter statt <code>array_rand</code> auf Rohdaten.
L5	Zeilen 159-186	Responsive Konfiguration (≈ 40 Zeilen) hardcodiert in einer Methode; Breakpoint-Liste als Konstante/Schleife (<code>foreach ([1400=>1920,...])</code>) wäre kürzer und prüfbarer. <code>setFormats(['jpg'=>['webp']])</code> liefert ausschließlich WebP ohne JPG-Fallback und greift nur für <code>jpg</code> -Quellen – bewusst? Sonst <code>['webp','jpg']</code> .
L6	<code>t1_page.php</code>	Palette per String-Concat; <code>PaletteManipulator</code> ist robust gegen fehlende Paletten und Contao-Standard. Logo erlaubt <code>jpg,svg,png</code> (kein <code>jpeg/webp/avif</code>), Hintergrund erlaubt mehr – vereinheitlichen. SQL als Doctrine-Array (<code>['type'=>'binary','length'=>16,'nullable'=>false]</code>) statt Strings (Contao-Konvention).

L7	Bundle/Plugin/Extension	<code>{@inheritdoc}</code> und Klassen-Docblock „Class X.“ sind Rauschen; <code>getPath()</code> -Override ist ab Symfony 7 / <code>AbstractBundle</code> überflüssig. Copyright „2023“ aktualisieren. <code>README.md</code> (liegt im Repo, im Git-Status als geändert) ist hier nicht Teil der Code-Analyse.
----	-------------------------	---

3. Sicherheit

- **XSS:** Freitext wird mit `htmlspecialchars(ENT_QUOTES, UTF-8, false)` ausgegeben – korrekt (Contao speichert Eingaben bereits entity-kodiert, daher `double_encode=false`). Bild-HTML stammt aus dem Contao-Image-Studio, nicht aus Nutzereingaben.
- **Pfadzugriff:** `is_readable($projectDir.'/'.$file->path)` nutzt DB-Pfade aus `tl_files`, kein Nutzer-Input → kein Traversal-Risiko.
- **Host-Header:** `HTTP_HOST` steuert nur die Auswahl zwischen vorhandenen Root-Seiten (Fallback: erste) – kein Sicherheitsproblem, aber Information-Leak-frei, da Logo/Hintergrund ohnehin öffentlich unter `assets/images/` liegen.
- **Keine SQL-Queries** im Bundle, ausschließlich Contao-Models (parametrisiert).
- **SVG-Logo:** Auslieferung über Studio/Assets; Redakteure mit Dateimanager-Rechten können SVGs hochladen – übliches Contao-Risiko, hier nicht verschärft.

4. Konformität zu php-best-practices

Kategorie	Bewertung	Anmerkung
Type System	gut	<code>strict_types</code> , Typen überall; <code>null string</code> statt <code>?string</code> konsistent. Lücke: untypisierte <code>callable/array \$m</code> in Closures (<code>list<string></code> PHPDoc), Magic-Properties (M1).
Modern PHP (8.3)	gut	Promotion, <code>readonly</code> , <code>match</code> nicht nötig. Chance: <code>#[AsHook]</code> , <code>#[\Override]</code> bei Plugin/Extension-Methoden, typisierte Konstante <code>private const array TEMPLATES</code> .
PSR	gut / 1 Abweichung	PSR-4 korrekt; PSR-12: Konstruktor-Klammern und Array-Einrückung (php-cs-fixer-Diff).
SOLID	mittel	SRP: Eine Klasse mischt Root-Page-Ermittlung, Bild-Erzeugung (2x) und HTML-Manipulation → aufteilbar in <code>RootPageResolver</code> , <code>LoginImageFactory</code> , <code>LoginHtmlInjector</code> . DIP: statische Abhängigkeiten (M1).
Error Handling	schwach	H2; Rückgabewerte <code>null</code> statt Exceptions bei fehlenden Dateien sind vertretbar, aber unprotokolliert.
Performance	ok	Lädt <i>alle</i> Root-Pages je Aufruf (<code>findByType('root')</code>) – für den Login irrelevant. Doppelte FilesModel-Lookups (M1). Zufällige Bildwahl verhindert Caching der gesamten Seite nicht, da Backend ohnehin nicht gecacht wird.
Security	gut	Siehe Abschnitt 3.

5. Positiv

- Bewusster Verzicht auf Template-Override, gut dokumentiert in `CLAUDE.md`.
- Defensive Anker-Logik mit Warnung statt Fatal-Error.
- Saubere Bundle-Struktur (Plugin, Extension, PSR-4, `classmap` mit Excludes).
- Kleine Angriffsfläche, kein eigener Controller, keine Formulare, kein SQL.

6. Empfohlene Maßnahmen (priorisiert)

Prio	Maßnahme	Aufwand
1	H1 beheben (Frontend-Link-Verlust)	5 min
2	H2: Try/Catch um Hook-Logik, Fehler loggen, Original-Buffer zurückgeben	10 min
3	M1: <code>fromFilesModel()</code> , <code>ContaoFramework/RequestStack</code> , PHPStan Level 9 auf 0 Fehler bringen	1–2 h
4	M2: CSS nur im Login laden	30 min
5	M3: Symfony-Translations de/en	30 min
6	M5 + T1: <code>phpstan/ecs/phpunit</code> einrichten; <code>doctrine-cache-bundle</code> entfernen	1 h
7	L1–L7 Aufräumen (final, <code>#[AsHook]</code> , <code>PaletteManipulator</code> , Docblocks)	1 h

T1 Testempfehlung

Unit-Tests (PHPUnit) für die reine String-Logik: Fixtures mit echtem `be_login`- und `be_login_two_factor`-HTML des aktuellen Cores; Fälle: alle Anker vorhanden, `</main>` fehlt (H1), `<body class=...>` / ohne class / `data-class`, Text `"0"`, Sonderzeichen im Text. Dafür muss die Injektions-Logik (Regex/Anker) in eine separate, framework-freie Klasse ausgelagert werden – auch das ist der Hauptnutzen der SRP-Aufteilung. Ein Anker-Smoke-Test gegen das installierte Core-Template fängt Markup-Änderungen bei Updates ab.

Geprüfte Dateien: `src/ (4)`, `config/services.yaml`, `contao/config/config.php`, `contao/dca/tl_page.php`, `contao/languages/de/tl_page.php`, `public/css/contao-login-screen.css`, `composer.json`. Es wurden keine Änderungen am Code vorgenommen.